

QuickBite Restaurant Ordering System

Final Project Report for a Multitier Business Application

Project domain	Restaurant menu, cart, and order management
Architecture	React frontend, Express backend, MongoDB database
Main roles	Customer and Admin
Currency	Polish zloty (PLN)
Prepared for	Multitier Business Application Development

Abstract

QuickBite is a multitier restaurant ordering and menu management system built as a practical business application. The project demonstrates a clear separation between the presentation tier, application tier, and data tier. Customers can register, log in, browse restaurant products, add products to a personal cart, view the total price in PLN, and place checkout orders. Admin users can manage menu products, view all orders, and update order status.

The system uses React and Vite for the frontend, Node.js and Express for the backend API, and MongoDB with Mongoose for persistence. It also includes role-based access control, token-based authentication, password hashing, password reset, request logging, rate limiting, analytics, real-time notifications, seed data, API documentation, and a third-party API integration through Open-Meteo weather data.

1. Introduction

Modern restaurants need digital systems that can present menus, manage products, accept customer orders, and support administrative control. Manual menu management is slow, difficult to update, and does not provide a clear overview of customer activity. QuickBite addresses this problem by providing a web-based restaurant system that works as a full-stack business application rather than a static website.

The project was designed to satisfy the requirements of a multitier business application. It separates the user interface from server-side business logic and stores data in a persistent database. This structure improves maintainability because each layer has a clear responsibility. The frontend focuses on interaction and presentation, the backend manages application rules and API requests, and the database stores users, foods, and orders.

QuickBite is also designed as a portfolio-style project. It includes visible user-facing features, backend APIs, documentation, and project evidence that can be demonstrated during a presentation. The goal is not only to create a working restaurant page, but to show architecture, data management, security, integration, and testing decisions in a complete project.

2. Problem Statement and Objectives

Restaurants often need to update menus, display prices, organize products by category, and track customer orders. Without a centralized application, these tasks can become inconsistent and difficult to manage. QuickBite solves this by offering a single web system for customers and administrators.

The main objective is to build a working restaurant ordering system using a multitier architecture. Customers should be able to browse a professional menu, select foods, build a personal cart, see the total cost, and place an order. Administrators should be able to add, edit, and delete menu items, and manage incoming orders.

The secondary objective is to satisfy academic requirements such as authentication, authorization, CRUD operations, persistent data storage, filtering, sorting, pagination, third-party API integration, real-time behavior, security measures, testing, and documentation. The final system must therefore show both practical usability and technical completeness.

3. Multitier Architecture

QuickBite follows a three-tier architecture. The presentation tier is the React frontend that runs in the browser. It displays the restaurant menu, authentication forms, analytics cards, cart, checkout, and order tracking sections. It communicates with the backend through HTTP requests using Axios and listens for live notifications through Server-Sent Events.

The application tier is the Node.js and Express backend. This layer exposes REST API endpoints, handles validation, authentication, authorization, password hashing, rate limiting, logging, analytics, and order workflow logic. It acts as the bridge between the frontend and the database.

The data tier is MongoDB Atlas accessed through Mongoose models. The database stores users, foods, and orders. Mongoose schemas define structure and validation rules for the stored documents. This makes the system persistent and allows the project to demonstrate related entities and query operations.

4. Technology Stack

The frontend uses React because it is suitable for interactive user interfaces and component-based application design. Vite is used as the development tool because it provides a fast local development server. Axios is used to communicate with the backend API.

The backend uses Node.js with Express. Express provides a simple and flexible API layer for REST endpoints. The backend is responsible for business logic, authentication, role checks, order creation, analytics, and integration with external services.

The database is MongoDB Atlas with Mongoose. MongoDB is appropriate for this project because menu products, users, and orders can be represented naturally as documents. Mongoose adds schema validation, model definitions, indexes, and a cleaner interface for data operations.

Layer	Technology	Purpose
Presentation	React, Vite, Axios	Responsive UI and API communication
Application	Node.js, Express	Business logic, API routing, auth, analytics
Data	MongoDB Atlas, Mongoose	Persistent users, foods, and orders
Integration	Open-Meteo, Server-Sent Events	External data and live updates

5. Functional Implementation

The menu module displays foods in categories: Meals, Soups, Desserts, and Drinks. Each product has a name, price, category, image, and optional descriptive data. Prices are displayed in PLN to match the Polish context. The menu includes restaurant-style product cards with images and clear pricing.

The authentication module supports registration, login, logout, and password reset. A user can register with simple credentials, log in, and then access user-specific functionality such as the personal cart and order history. The first account can become admin, and a demo admin account can be used with admin/admin.

Role-based authorization is implemented so that normal customers do not see admin controls. Add, edit, and delete buttons are visible only when an admin is logged in. This improves the user experience and

also demonstrates authorization at the interface level. The backend also protects admin routes, so hiding the buttons is not the only security measure.

The cart module gives each logged-in account its own personal cart. Users can add products, increase or decrease quantity, remove items, clear the cart, and view the total price. Cart data is kept separately per account in browser storage for demonstration purposes.

The checkout module converts the cart into a saved order. A customer can place an order, after which the order is stored in MongoDB. Customers can view their own orders, while admin users can view all orders. Admins can update order status to Preparing, Ready, or Completed. This creates a clear business workflow beyond basic product management.

Analytics cards show information such as total foods, total orders, and average price. This supports the requirement for reporting or analytics features and gives the application a more complete business feel.

Real-time notifications are implemented with Server-Sent Events. When menu items or orders are changed, the backend sends a live update message to connected clients. This satisfies the requirement for a real-time element without requiring a heavy WebSocket setup.

Third-party API integration is implemented through Open-Meteo. The backend requests live Warsaw weather data and the frontend displays it as a delivery demand planning card. This demonstrates external API communication and error handling.

6. Data Management

The application uses three main entities: User, Food, and Order. The User model stores account information, hashed password, role, and password reset data. The Food model stores restaurant products, categories, prices, images, availability, rating, and preparation time. The Order model stores customer orders, order items, total price, status, and timestamps.

The /foods endpoint supports querying, filtering, sorting, and pagination. A request can filter foods by category, search by name, sort by fields such as name or price, and request paginated results. This directly addresses the data management requirements for querying, filtering, sorting, and pagination.

Seed data is included so the project can quickly populate the database with realistic restaurant items. The seed data includes Meals, Soups, Desserts, and Drinks with realistic PLN prices. This makes demonstrations easier and ensures the application does not appear empty on first use.

Orders are saved in MongoDB so checkout activity becomes persistent. Each order is connected to a user and contains item-level details. This shows a related-entity workflow and makes the project stronger as a business application.

7. Security and Performance

Passwords are hashed before storage using a deterministic hashing function in the demo backend. The system also supports migration of older plain-text demo passwords by converting them during login. In a production system, bcrypt or argon2 would be recommended, but the project still demonstrates the principle that raw passwords should not be stored.

The backend generates a signed token for authenticated users. Protected routes check the token before returning profile or order data. Admin routes also check the user role before allowing menu changes or order status changes.

Input validation is applied in the main API routes. For example, foods require a valid name and positive price, orders require valid items and quantities, and password reset requires account and password data. Invalid requests return clear error messages.

The backend includes request logging and simple rate limiting. Logging records request method, URL, status code, and response time. Rate limiting helps protect the API from repeated requests. Performance is also improved through pagination and limited query responses.

8. Testing Strategy

The project includes sample backend tests using Node's built-in test runner. The tests verify password hashing behavior and order total calculation. These tests demonstrate unit testing for security-related utility logic and business calculation logic.

A full production-level project would expand testing further. Recommended additions include integration tests for authentication endpoints, order endpoints, and food CRUD endpoints, plus end-to-end tests for browser workflows such as login, add to cart, checkout, and admin status update.

The academic requirement mentions more than 70 percent test coverage. The current project includes test examples and a documented testing strategy, but the coverage target should be described honestly as a future improvement unless a full suite is added. For a student project demonstration, the existing tests still show awareness of testing responsibilities.

9. Documentation and Presentation

The project includes a README with setup instructions, user guide, architecture diagram, technology stack, and requirements coverage. It also includes API documentation explaining authentication, food endpoints, order endpoints, analytics, weather integration, and real-time events.

For a 10 to 15 minute demo, the suggested flow is: show the architecture diagram, start backend and frontend, log in as a customer, add items to the cart, place an order, log in as admin, update order status, show analytics, and briefly explain the Open-Meteo and real-time notification features.

The GitHub repository should include frontend, backend, models, tests, README, API documentation, seed scripts, and report files. Sensitive configuration such as the real MongoDB connection string should remain in .env and should not be committed.

Suggested Demo Flow

1. Start the backend and frontend.
2. Show the architecture diagram in the README.
3. Register or log in as a customer.
4. Add foods to the cart and show the total in PLN.
5. Place an order and show it in the order tracking panel.
6. Log in as admin/admin.
7. Add, edit, or delete a menu product.
8. Update order status and show the live notification.

9. Show analytics and Open-Meteo weather integration.

10. Limitations and Future Improvements

The current cart is stored per account in browser local storage. This is useful for a demo, but a production version should store carts in the database so cart state is shared across devices and browsers.

The password hashing implementation demonstrates the concept, but a production application should use bcrypt or argon2 with salts. JWT handling could also be replaced with a standard library for stronger production security.

The project can be improved with Stripe payment simulation, email-based password reset, full test coverage, Cypress end-to-end tests, admin charts, PDF order receipts, PWA support, internationalization, and deployment to platforms such as Vercel and Render.

Requirements Coverage Matrix

Requirement	Status	Evidence
Multitier architecture	Implemented	React frontend, Express backend, MongoDB database
Authentication	Implemented	Register, login, logout, password reset
Authorization	Implemented	Admin/customer roles and protected API routes
Business logic	Implemented	Menu CRUD, cart, checkout, order status workflow
Data management	Implemented	User, Food, Order models; seed data; queries
Filtering/sorting/pagination	Implemented	/foods query parameters
Responsive UI	Implemented	Restaurant menu, cart, analytics, order panels
Third-party API	Implemented	Open-Meteo weather integration
Real-time element	Implemented	Server-Sent Events live notifications
Security/performance	Implemented	Password hashing, token validation, logging, rate limiting
Testing	Partially implemented	Unit tests included; full coverage remains a future improvement
Documentation	Implemented	README, API docs, report, architecture diagram

API Summary

Endpoint	Purpose
POST /register	Creates a user and returns token/user data

Endpoint	Purpose
POST /login	Authenticates user and returns token/user data
POST /password-reset	Resets a demo account password
GET /foods	Returns menu items with filtering, sorting, pagination
POST /foods	Admin-only menu item creation
PUT /foods/:id	Admin-only menu item update
DELETE /foods/:id	Admin-only menu item deletion
POST /orders	Creates checkout order from cart
GET /orders	Returns customer orders or all orders for admin
PUT /orders/:id/status	Admin-only order status update
GET /analytics	Returns totals and averages
GET /weather	Returns Open-Meteo weather data
GET /events	Provides live notification stream

11. Conclusion

QuickBite successfully demonstrates a multitier business application for restaurant menu and order management. It includes a professional responsive frontend, an Express backend, and MongoDB persistence. The project implements authentication, authorization, admin controls, menu CRUD, personal carts, checkout orders, order status tracking, analytics, external API integration, real-time notifications, documentation, and tests.

The application meets the main goal of showing practical full-stack development with clear separation of concerns. It can be presented as a working business system and also as a portfolio project that demonstrates architecture, security, database design, and user experience.

References

- React Documentation
- Node.js Documentation
- Express.js Documentation
- MongoDB and Mongoose Documentation
- Open-Meteo API Documentation
- Course project requirements for multitier business applications